```
import heapq

class PuzzleState:
    def __init__(self, board, g=0, parent=None, action="Initial"):
        self.board = tuple(board)
        self.g = g  # Path cost (depth)
        self.parent = parent
        self.action = action
        self.h = 0  # Heuristic value

    @property
    def f(self):
        return self.g + self.h

    def __lt__(self, other):
        return self.f < other.f

    def get_blank_pos(self):
        idx = self.board.index(0)
        return idx // 3, idx % 3

    def get_neighbors(self):
        neighbors = []
        r, c = self.get_blank_pos()
        moves = {
            "Up": (r - 1, c),
            "Down": (r + 1, c),
            "Left": (r, c - 1),
            "Right": (r, c + 1)
        }

        for action, (nr, nc) in moves.items():
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_blank_idx = nr * 3 + nc
                old_blank_idx = r * 3 + c

                new_board = list(self.board)
                new_board[old_blank_idx], new_board[new_blank_idx] = new_board[new_blank_idx], new_board[old_blank_idx]

                neighbors.append(PuzzleState(new_board, self.g + 1, self, action))
        return neighbors

    def print_board(self):
        for i in range(0, 9, 3):
            row = [str(x) if x != 0 else "_" for x in self.board[i:i+3]]
            print(" ".join(row))
        print()

# Heuristic: Count of misplaced tiles (excluding the blank space)
def heuristic_misplaced_tiles(board, goal_board):
    count = 0
    for i in range(len(board)):
        if board[i] != 0 and board[i] != goal_board[i]:
            count += 1
    return count

# Core A* Search Function
def a_star_search(initial_board, goal_board):
    start_state = PuzzleState(initial_board, g=0)
    start_state.h = heuristic_misplaced_tiles(start_state.board, goal_board)

    frontier = []
    heapq.heappush(frontier, start_state)
    explored = set()

    while frontier:
        current_state = heapq.heappop(frontier)

        if current_state.board == tuple(goal_board):
            return current_state

        explored.add(current_state.board)

        for neighbor in current_state.get_neighbors():
            if neighbor.board in explored:
```

```
                continue

            neighbor.h = heuristic_misplaced_tiles(neighbor.board, goal_board)
            heapq.heappush(frontier, neighbor)

    return None

def print_solution_path(solution_node):
    path = []
    curr = solution_node
    while curr:
        path.append(curr)
        curr = curr.parent
    path.reverse()

    print(f"🎉 Goal reached in {len(path) - 1} moves!\n")
    for step, state in enumerate(path):
        print(f"Step {step}: Action taken -> {state.action} (g={state.g}, h={state.h}, f={state.f})")
        state.print_board()

# --- Execution ---
if __name__ == "__main__":
    # 0 represents the empty block '_'
    initial_state = [
        2, 8, 3,
        1, 6, 4,
        0, 7, 5
    ]

    goal_state = [
        1, 2, 3,
        8, 0, 4,
        7, 6, 5
    ]

    print("====================================================")
    print("🤖 Running A* Search (Misplaced Tiles Heuristic Case)")
    print("====================================================")
    solution = a_star_search(initial_state, goal_state)
    if solution:
        print_solution_path(solution)
```

```
====================================================
🤖 Running A* Search (Misplaced Tiles Heuristic Case)
====================================================
🎉 Goal reached in 6 moves!

Step 0: Action taken -> Initial (g=0, h=5, f=5)
2 8 3
1 6 4
_ 7 5

Step 1: Action taken -> Right (g=1, h=4, f=5)
2 8 3
1 6 4
7 _ 5

Step 2: Action taken -> Up (g=2, h=3, f=5)
2 8 3
1 _ 4
7 6 5

Step 3: Action taken -> Up (g=3, h=3, f=6)
2 _ 3
1 8 4
7 6 5

Step 4: Action taken -> Left (g=4, h=2, f=6)
_ 2 3
1 8 4
7 6 5

Step 5: Action taken -> Down (g=5, h=1, f=6)
1 2 3
_ 8 4
7 6 5

Step 6: Action taken -> Right (g=6, h=0, f=6)
1 2 3
8 _ 4
```

```
7 6 5
```

```
import itertools

def get_attacking_pairs(state):
    """
    Cost Function: Counts the number of pairs of queens attacking each other
    directly or indirectly (checks rows and diagonals).
    """
    attacks = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            # Same row attack
            if state[i] == state[j]:
                attacks += 1
            # Diagonal attack: row difference == column difference
            elif abs(state[i] - state[j]) == abs(i - j):
                attacks += 1
    return attacks

def get_best_neighbor(state):
    """
    Neighbor Generation: Swaps the row positions of two queens.
    Enforces the tie-breaking rule using sorted column indices (i, j).
    """
    n = len(state)
    best_state = None
    min_cost = float('inf')
    best_swap_indices = (float('inf'), float('inf'))

    print("\n--- Generating and evaluating all swap neighbors ---")

    # Generate pairs of columns to swap (i, j) where i < j
    # This automatically processes pairs in order of smallest subscripts first
    for i, j in itertools.combinations(range(n), 2):
        # Create neighbor state by swapping values at index i and j
        neighbor = list(state)
        neighbor[i], neighbor[j] = neighbor[j], neighbor[i]

        cost = get_attacking_pairs(neighbor)
        print(f"Swap columns ({i}, {j}) -> Neighbor State: {neighbor} | Cost: {cost}")

        # Check if this neighbor is strictly better or resolves a tie-breaker
        if cost < min_cost:
            min_cost = cost
            best_state = neighbor
            best_swap_indices = (i, j)
        elif cost == min_cost:
            # If cost is tied, pick the one with smaller subscripts (i, j lexicographically)
            if (i, j) < best_swap_indices:
                best_state = neighbor
                best_swap_indices = (i, j)

    return best_state, min_cost, best_swap_indices

def print_board(state):
    """Helper to visualize the 4x4 chessboard configurations"""
    n = len(state)
    for row in range(n):
        line = ""
        for col in range(n):
            if state[col] == row:
                line += " Q "
            else:
                line += " . "
        print(line)
    print()

def hill_climbing_4_queens(initial_state):
    """Core Hill Climbing algorithm execution block"""
    current_state = list(initial_state)
    current_cost = get_attacking_pairs(current_state)

    step = 0
    print("================================================")
```

```
    print("🚀 STARTING HILL CLIMBING (4-QUEENS)")
    print("==================================================")
    print(f"Step {step} (Initial State): {current_state} | Cost: {current_cost}")
    print_board(current_state)

    while True:
        # Fetch the optimal candidate from neighboring mutations
        neighbor_state, neighbor_cost, swap_used = get_best_neighbor(current_state)

        # Terminate search if neighbor cost doesn't strictly improve over current value
        if neighbor_cost >= current_cost:
            print("\n==================================================")
            if current_cost == 0:
                print("🎉 SUCCESS: Global optimum goal state discovered!")
            else:
                print("🔴 TERMINATED: Hit local maxima/plateau. No strict improvements available.")
            print("==================================================")
            print(f"Final State Configuration: {current_state} with final cost: {current_cost}")
            print_board(current_state)
            break

        # Move up the gradient to the superior neighboring state
        step += 1
        current_state = neighbor_state
        current_cost = neighbor_cost

        print(f"\n➔ Step {step}: Selected swap {swap_used} -> State: {current_state} | Cost: {current_cost}")
        print_board(current_state)

if __name__ == "__main__":
    # Pre-defined setup mandated by your Observation Book (Page 5)
    # x0 = 3, x1 = 1, x2 = 2, x3 = 0
    lab_initial_state = [3, 1, 2, 0]

    hill_climbing_4_queens(lab_initial_state)
```

```
==================================================
🚀 STARTING HILL CLIMBING (4-QUEENS)
==================================================
Step 0 (Initial State): [3, 1, 2, 0] | Cost: 2
 .  .  .  Q
 .  Q  .  .
 .  .  Q  .
 Q  .  .  .


--- Generating and evaluating all swap neighbors ---
Swap columns (0, 1) -> Neighbor State: [1, 3, 2, 0] | Cost: 1
Swap columns (0, 2) -> Neighbor State: [2, 1, 3, 0] | Cost: 1
Swap columns (0, 3) -> Neighbor State: [0, 1, 2, 3] | Cost: 6
Swap columns (1, 2) -> Neighbor State: [3, 2, 1, 0] | Cost: 6
Swap columns (1, 3) -> Neighbor State: [3, 0, 2, 1] | Cost: 1
Swap columns (2, 3) -> Neighbor State: [3, 1, 0, 2] | Cost: 1

➔ Step 1: Selected swap (0, 1) -> State: [1, 3, 2, 0] | Cost: 1
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .
 .  Q  .  .


--- Generating and evaluating all swap neighbors ---
Swap columns (0, 1) -> Neighbor State: [3, 1, 2, 0] | Cost: 2
Swap columns (0, 2) -> Neighbor State: [2, 3, 1, 0] | Cost: 2
Swap columns (0, 3) -> Neighbor State: [0, 3, 2, 1] | Cost: 4
Swap columns (1, 2) -> Neighbor State: [1, 2, 3, 0] | Cost: 4
Swap columns (1, 3) -> Neighbor State: [1, 0, 2, 3] | Cost: 2
Swap columns (2, 3) -> Neighbor State: [1, 3, 0, 2] | Cost: 0

➔ Step 2: Selected swap (2, 3) -> State: [1, 3, 0, 2] | Cost: 0
 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .


--- Generating and evaluating all swap neighbors ---
Swap columns (0, 1) -> Neighbor State: [3, 1, 0, 2] | Cost: 1
Swap columns (0, 2) -> Neighbor State: [0, 3, 1, 2] | Cost: 1
Swap columns (0, 3) -> Neighbor State: [2, 3, 0, 1] | Cost: 4
```

```
Swap columns (1, 2) -> Neighbor State: [1, 0, 3, 2] | Cost: 4
Swap columns (1, 3) -> Neighbor State: [1, 2, 0, 3] | Cost: 1
Swap columns (2, 3) -> Neighbor State: [1, 3, 2, 0] | Cost: 1

==================================================
🎉 SUCCESS: Global optimum goal state discovered!
==================================================
Final State Configuration: [1, 3, 0, 2] with final cost: 0
 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .
```

```
import math
import random

def get_conflicts(board):
    conflicts = 0
    n = len(board)
    for c1 in range(n):
        for c2 in range(c1 + 1, n):
            r1 = board[c1]
            r2 = board[c2]
            if r1 == r2:
                conflicts += 1
            elif abs(r1 - r2) == abs(c1 - c2):
                conflicts += 1
    return conflicts

def generate_neighbor(board):
    new_board = list(board)
    col = random.randint(0, 7)
    current_row = board[col]
    while True:
        new_row = random.randint(0, 7)
        if new_row != current_row:
            break
    new_board[col] = new_row
    return new_board, col, new_row

def print_chessboard(board):
    for row in range(8):
        line = ""
        for col in range(8):
            line += " Q " if board[col] == row else " . "
        print(line)
    print()

def simulated_annealing_8_queens():
    # Initial parameters from lab manual page 2
    current_board = [0, 1, 2, 3, 4, 5, 6, 7]
    T = 10.0
    alpha = 0.90

    current_cost = get_conflicts(current_board)

    print("==================================================")
    print("🔥 STARTING SIMULATED ANNEALING (8-QUEENS)")
    print("==================================================")
    print(f"Initial State: {current_board} | Cost: {current_cost} conflicts | Temp: {T}\n")

    iteration = 1
    # Loop continues until a solution is found or the temperature cools down completely
    while T > 0.0001 and current_cost > 0:
        proposed_board, col, new_row = generate_neighbor(current_board)
        proposed_cost = get_conflicts(proposed_board)
        delta_E = proposed_cost - current_cost

        # Only print details for the first 3 steps to keep output short and match the manual
        show_log = iteration <= 3

        if show_log:
            print(f"### Step {iteration}")
            print(f"Action: Randomly pick Column {col} and move queen to row {new_row}")
            print(f"Proposed Board: {proposed_board} | Proposed Cost: {proposed_cost}")
            print(f"Cost Difference: ΔE = {delta_E}")
```

```
            # Acceptance Check
            if delta_E < 0:
                current_board = proposed_board
                current_cost = proposed_cost
                if show_log:
                    print("Decision: ACCEPT move immediately (improvement).")
            else:
                p = math.exp(-delta_E / T)
                r = random.random()
                if show_log:
                    print(f"Acceptance Calculation: P = e^(-{delta_E} / {T:.2f}) ≈ {p:.3f}")
                    print(f"Random Roll: R = {r:.2f}")

                if r < p:
                    current_board = proposed_board
                    current_cost = proposed_cost
                    if show_log:
                        print("Decision: ACCEPT the worse move to escape local minima.")
                else:
                    if show_log:
                        print("Decision: REJECT the move and revert board.")

            T = T * alpha
            if show_log:
                print(f"Temperature Update: T = {T:.2f}\n")
                if iteration == 3:
                    print("... [Suppressing intermediate steps for clean output] ...\n")

            iteration += 1

        print("================================================")
        print("📉 CONVERGENCE SUMMARY")
        print("================================================")
        print(f"Final Board: {current_board}")
        print(f"Final Cost: {current_cost} (Solution Found)" if current_cost == 0 else f"Final Cost: {current_cost} conflicts")
        print("\nVisual Board Matrix:")
        print_chessboard(current_board)

    if __name__ == "__main__":
        # Optional seed to reliably find a successful solution in a single run
        random.seed(42)
        simulated_annealing_8_queens()
```

```
================================================
🔥 STARTING SIMULATED ANNEALING (8-QUEENS)
================================================
Initial State: [0, 1, 2, 3, 4, 5, 6, 7] | Cost: 28 conflicts | Temp: 10.0

### Step 1
Action: Randomly pick Column 1 and move queen to row 0
Proposed Board: [0, 0, 2, 3, 4, 5, 6, 7] | Proposed Cost: 22
Cost Difference: ΔE = -6
Decision: ACCEPT move immediately (improvement).
Temperature Update: T = 9.00

### Step 2
Action: Randomly pick Column 4 and move queen to row 3
Proposed Board: [0, 0, 2, 3, 3, 5, 6, 7] | Proposed Cost: 18
Cost Difference: ΔE = -4
Decision: ACCEPT move immediately (improvement).
Temperature Update: T = 8.10

### Step 3
Action: Randomly pick Column 3 and move queen to row 2
Proposed Board: [0, 0, 2, 2, 3, 5, 6, 7] | Proposed Cost: 15
Cost Difference: ΔE = -3
Decision: ACCEPT move immediately (improvement).
Temperature Update: T = 7.29

... [Suppressing intermediate steps for clean output] ...

================================================
📉 CONVERGENCE SUMMARY
================================================
Final Board: [3, 1, 4, 2, 0, 6, 7, 0]
Final Cost: 2 conflicts

Visual Board Matrix:
 .  .  .  .  Q  .  .  Q
```

```
. Q . . . . . .
. . . Q . . . .
Q . . . . . . .
. . Q . . . . .
. . . . . . . .
. . . . . Q . .
. . . . . . Q .
```